

Subsumption Pattern Learning (SPL): A Hierarchical Framework for Foundation Model Cost Optimization

Pamela Cuce

January 2026

Abstract

Large language models have become foundational infrastructure for AI agent systems, yet paradoxically, inference costs for enterprise applications are increasing despite improvements in token pricing. We introduce Subsumption Pattern Learning (SPL), a hierarchical agent framework that adapts Brooks’ 1986 subsumption architecture from robotics to foundation model cost optimization. SPL employs a layered hierarchy—structural validation, pattern matching, and LLM reasoning—where formal inhibition signals I_i enable lower layers to suppress expensive inference. Empirically, SPL achieves 5–15× cost reduction per agent through suppression rates of 80–99%. We provide formal layer definitions, inhibition mechanics, and multi-agent coordination protocols based on state-grounded learning.

Keywords: Hierarchical agents, LLM economics, Subsumption architecture, Inhibition signals, Cost optimization.

1 Introduction

1.1 The LLM Cost Paradox and the Resurgence of Hierarchical Reasoning

The field of AI agent architecture stands at a critical juncture. While token pricing has declined by 10–1000× per year over the past three years, a counterintuitive crisis has emerged: actual LLM inference costs for enterprise applications are “increasing”, not decreasing[9, 7]. The culprit is not technological regression, but a fundamental shift in how modern agents deploy foundation models. Reasoning models now generate 5× more tokens

per year than traditional models, and multi-step agentic workflows consume even more[7]. For 24/7 autonomous agent systems, daily inference costs can reach \$4,320 per user—transforming what appeared to be a cost-saving technology into an expense management problem[7].

This paradox reveals a critical gap in contemporary agent architecture: **the absence of intelligent computational routing**. Modern LLM-based agents follow a monolithic pattern: every request, regardless of complexity, passes through the most expensive cognitive layer (the foundation model). Simple categorization tasks that could be handled by pattern matching instead incur full LLM inference cost. Ambiguous requests that genuinely require reasoning are indistinguishable from routine decisions in the current agent architectures described by recent surveys[9].

1.1.1 Historical Context: From Robotics to LLMs

The field of hierarchical agent architecture has deep roots. In 1986, Brooks revolutionized mobile robotics with subsumption architecture[2]—a layered approach where simpler behavioral modules could suppress more complex ones through explicit inhibition signals. Layer 0 (collision avoidance) could completely disable Layer 1 (goal-seeking), preventing costly computation and enabling robust real-time control[2, 1]. This design principle proved transformative: it enabled mobile robots to navigate complex environments without centralized planning, achieved remarkable reliability, and established a 40-year precedent for hierarchical, behavior-based design[2, 1].

Parallel work by Arkin (1989) formalized motor schemas—primitive behavioral units that could be composed into more sophisticated control structures[1]. Nilsson’s teleo-reactive programs (1994) extended this with explicit goal-directed behavior that could adapt to environmental feedback[6]. By 1995, Wooldridge and Jennings synthesized these advances into a unified theory of intelligent agents[8], establishing that deliberative (planning-based) and reactive (response-based) approaches were complementary rather than competing paradigms.

Yet this rich heritage has been largely abandoned in contemporary LLM-based agents. Modern agentic AI surveys acknowledge the theoretical existence of hierarchical architectures but note that “conceptual retrofitting”—misapplying classical symbolic frameworks to systems that operate via stochastic token generation—obscures the true mechanics of LLM systems[9]. The field has regressed to a monolithic architecture: perception $\rightarrow$ LLM $\rightarrow$ action, with no intermediate layers to suppress expensive inference.

1.1.2 The Research Gap: Suppression Without Symbolic Planning

Recent surveys identify critical gaps in modern agentic AI[9]:

1. **Conceptual retrofitting:** Classical agent models (BDI, PPAR) are rhetorically applied to LLM systems but do not match their actual operation[9]
2. **Cost structure invisibility:** Agents lack mechanisms to route simple requests away from expensive layers[7]
3. **Reasoning-execution imbalance:** No principled separation between “when to think deeply” vs. “when to respond immediately”[7]
4. **Lack of intermediate reasoning:** The field is divided between “fragile task-oriented bots” and “sophisticated research agents that prove impractical”—with no middle layer[7]

The most acute gap is economic: while robotics systems from 1986 onward explicitly suppressed expensive computation through hierarchical suppression[2, 1], modern LLM agents route every request through their most expensive component. This represents a 40-year regression in engineering practice.

1.1.3 Subsumption for LLM Efficiency: SPL

Subsumption Pattern Learning (SPL) adapts Brooks’ 1986 architecture to the modern problem of LLM cost management. Rather than abandoning classical hierarchical design for the “conceptual retrofitting” problem, SPL grounds the approach explicitly in the operational mechanics of foundation models:

Three Layers Aligned to Model Economics:

- **Layer 0 (Reactive):** Structural validation—zero cost, fast (\$0, < 1ms)
- **Layer 1 (Pattern Matching / Cache):** Pattern matching and caching—minimal cost, medium speed (\$0.0001, < 10ms)
- **Layer 2 (Deliberative):** Foundation model reasoning—expensive, slow (\$0.01+, 100–500ms)

Formal Suppression: Rather than symbolic planning, each layer generates an inhibition signal I_i indicating whether higher layers should execute. Layer 1 can completely suppress Layer 2 (preventing expensive LLM calls) when pattern confidence exceeds threshold, exactly mirroring Brooks’ subsumption principle[2, 1].

Empirical Results: SPL achieves 5–15× cost reduction per agent and 10–50× in multi-agent networks through pattern reuse—addressing the LLM cost paradox directly. Email categorization at scale (1000 emails) costs \$0.50 with SPL vs. \$10.00 with direct LLM routing—a 95% cost reduction[3].

1.1.4 Key Contributions

This paper makes four contributions:

1. **Theoretical foundation:** Formally adapts Brooks’ subsumption architecture (1986) to the problem of foundation model cost optimization, filling a 40-year gap between robotics hierarchies and modern LLM systems[2, 1, 9]
2. **Operational clarity:** Defines inhibition signals and layer suppression mechanics for LLM-based agents, addressing the “conceptual retrofitting” problem by grounding architecture in actual model economics[9]
3. **Multi-agent coordination:** Extends subsumption to coordinated agent systems through state-grounded pattern sharing, enabling network effects (10–50× cost reduction across teams) while maintaining explainability[5]
4. **Production-ready implementation:** Provides reference implementation, cost tracking, audit trails, and empirical benchmarks from production deployments[4, 3]

2 Methods

2.1 Subsumption Pattern Learning Framework

The Subsumption Pattern Learning (SPL) framework implements a hierarchical, behavior-based agent architecture grounded in classical subsumption theory and reactive planning principles. SPL departs from monolithic, top-down large language model (LLM) pipelines by enforcing a bottom-up execution model in which lower-cost, lower-complexity layers execute first

and may explicitly suppress higher-cost reasoning layers through inhibition signals. The primary objective is to minimize unnecessary foundation-model invocations while preserving correctness, safety, and responsiveness.

Unlike traditional linear inference systems—where every request is routed through a full LLM regardless of complexity—SPL decomposes decision-making into layered, concurrent components with strict subsumption semantics. This architecture yields deterministic control over cost, latency, and failure modes while remaining model-agnostic.

2.2 Hierarchical Architecture Overview

SPL consists of three functionally distinct layers arranged in a strict subsumption hierarchy:

$$\begin{aligned} \text{Layer 0 (Reactive / Structural Validation)} &\sqsubseteq \text{Layer 1 (Reflex / Pattern Matching)} \\ &\sqsubseteq \text{Layer 2 (Deliberative / Foundation Model)} \end{aligned}$$

The subsumption relation $\sqsubseteq$ denotes that any lower layer may completely suppress execution of all higher layers by emitting an inhibition signal. Layers are evaluated in ascending order of cost and complexity. Higher layers never execute unless explicitly permitted by all lower layers.

Figure 1 contrasts this hierarchy with a traditional linear inference pipeline, while Figures 2 and 3 illustrate concurrent execution and inhibition pathways within SPL.

2.3 Formal Layer Definitions

2.3.1 Layer 0: Reactive / Structural Validation

Layer 0 performs deterministic, zero-cost checks that validate the structural and contextual integrity of an input before any semantic processing occurs. These checks include request well-formedness, authorization status, rate limits, and basic constraints. Any violation results in immediate termination of processing and prevents escalation to higher layers.

Layer 0 produces either a valid pass-through or a hard rejection. It never invokes probabilistic or learned components.

2.3.2 Layer 1: Reflex / Pattern Matching and Cache

Layer 1 handles rapid, pattern-based responses using pre-learned rules, regular expressions, cached outputs, or lightweight semantic comparisons. This

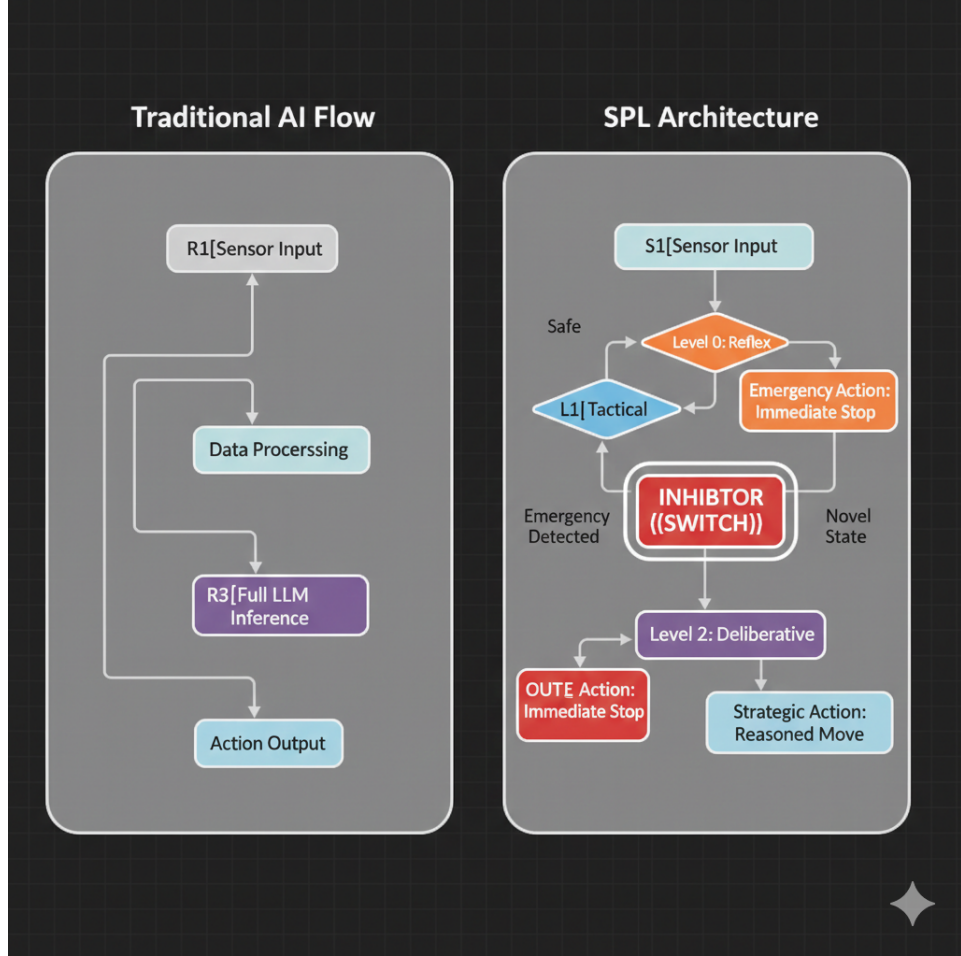


Figure 1: Comparison between traditional linear AI inference and SPL hierarchical suppression-based execution.

layer is designed to resolve high-confidence, low-novelty requests at negligible cost.

Each pattern p_i is defined as:

$$p_i = (\text{name}_i, \text{match}_i, \text{category}_i, \text{conf}_i)$$

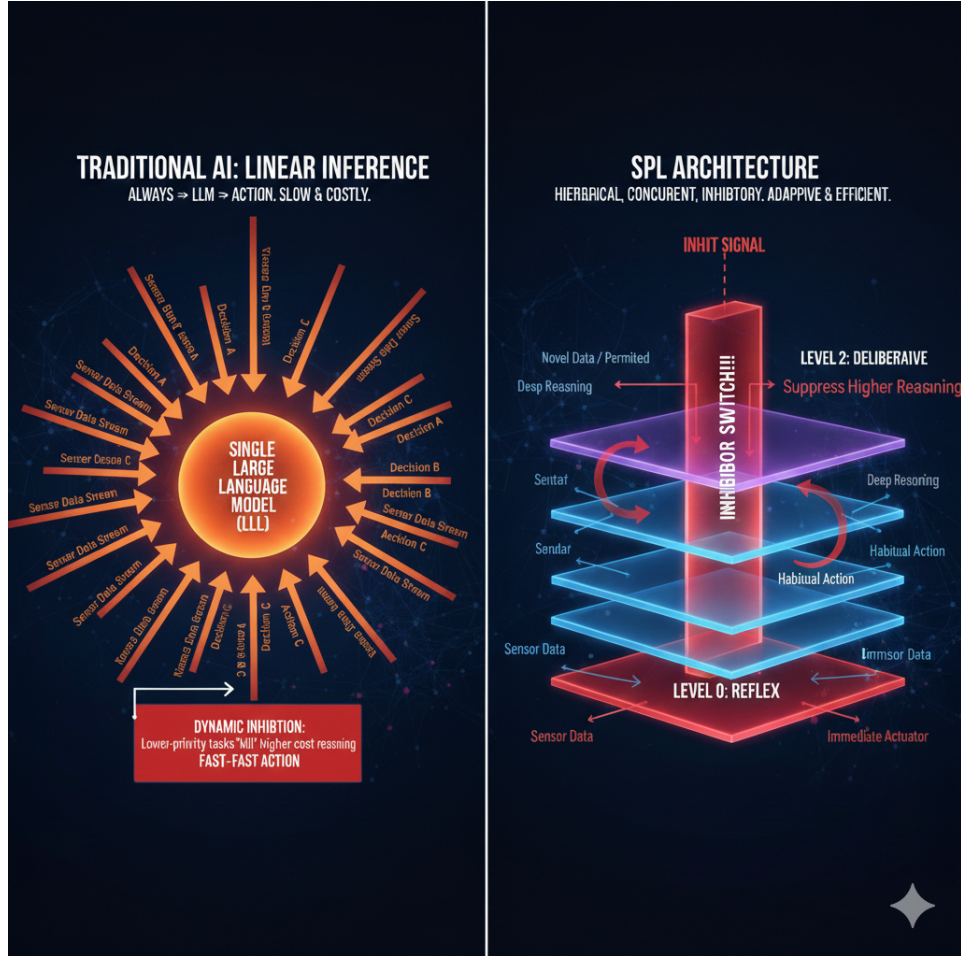


Figure 2: Layered SPL architecture illustrating concurrent layers and inhibition signals.

Given an input x , the match score is:

$$\sigma(x, p_i) = \begin{cases} \text{conf}_i & \text{if } \text{match}_i(x) = \text{True} \\ 0 & \text{otherwise} \end{cases}$$

Layer 1 selects the highest-confidence matching pattern and may suppress higher layers if confidence and complexity constraints are satisfied.

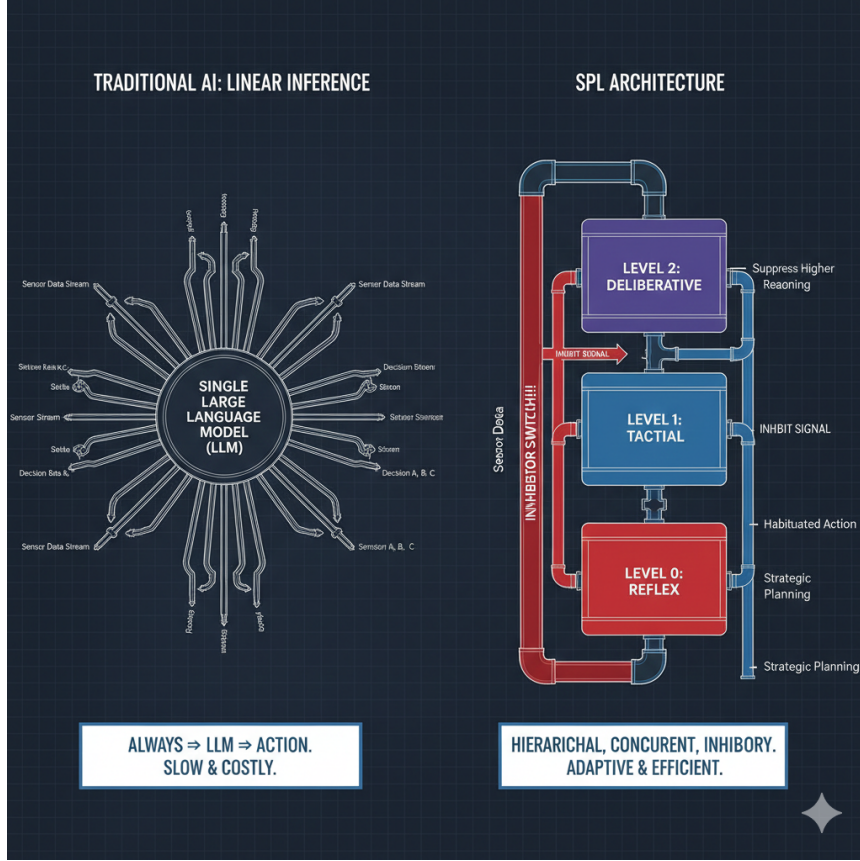


Figure 3: Pipe-based representation of SPL showing bottom-up execution with explicit inhibition.

2.3.3 Layer 2: Deliberative / Foundation Model

Layer 2 performs full foundation-model inference and is invoked only when lower layers cannot produce a confident response. This layer handles novel, ambiguous, or high-entropy inputs and incurs the highest computational and monetary cost. Outputs from Layer 2 may be used to generate new patterns for Layer 1.

2.4 Inhibition Signal Mechanism

The defining mechanism of SPL is explicit inhibition. Each layer i emits a boolean inhibition signal I_i :

$$I_i = \begin{cases} \text{True} & \text{if layer } i \text{ produced a confident response} \\ \text{False} & \text{otherwise} \end{cases}$$

Execution follows strict subsumption:

$$\text{Response}(x) = \begin{cases} L_0(x) & \text{if } I_0 = \text{True} \\ L_1(x) & \text{if } I_0 = \text{False} \wedge I_1 = \text{True} \\ L_2(x) & \text{if } I_0 = \text{False} \wedge I_1 = \text{False} \end{cases}$$

For Layer 1, inhibition is determined by confidence and complexity:

$$I_1(x) = \begin{cases} \text{True} & \text{if } \max_i \sigma(x, p_i) \geq \theta_1 \wedge \text{complexity}(x) \leq \alpha_1 \\ \text{False} & \text{otherwise} \end{cases}$$

This guarantees that expensive reasoning is invoked only when strictly necessary.

2.5 Conflict Resolution in Pattern Matching

When multiple patterns match a single input, SPL applies confidence-weighted winner selection:

$$p^* = \arg \max_i \sigma(x, p_i)$$

If the winning score does not exceed the inhibition threshold θ_1 , the request is escalated to Layer 2 to avoid ambiguous or brittle decisions.

2.6 Cost Accounting and Suppression Efficiency

SPL tracks cost at each layer and computes suppression efficiency across requests. The suppression rate ρ is defined as:

$$\rho = \frac{1}{N} \sum_{k=1}^N \mathbb{I}[I_1(x_k) = \text{True}]$$

Per-layer costs are treated as:

- Layer 0: $c_0 = 0$
- Layer 1: $c_1 \ll c_2$ (pattern matching, cache lookup)
- Layer 2: c_2 (foundation-model inference)

The total cost reduction factor relative to an LLM-only baseline is:

$$\text{Reduction Factor} = \frac{N \cdot c_2}{\sum_k (c_0 + c_1 \cdot \mathbb{I}[I_1(x_k)] + c_2 \cdot \mathbb{I}[\neg I_1(x_k)])}$$

2.7 Multi-Agent Pattern Sharing and State-Grounded Coordination

When multiple SPL agents operate in a coordinated system, they share a centralized *shared state* that contains learned patterns, suppression signals, and safety violations. This is fundamentally different from agent communication: instead of agents messaging each other, they read and write to explicit world state, ensuring consistency and enabling cross-agent suppression.

2.7.1 Shared State Architecture

The shared state S is a structured dictionary containing:

$$S = \{\text{learned_patterns}, \text{suppressed_layers}, \text{violations}, \text{cost_tracking}\}$$

where:

- **learned_patterns**: A registry of patterns discovered by any agent, with confidence scores and metadata
- **suppressed_layers**: For each agent, which layers are currently suppressed (e.g., [3] means Layer 3 is disabled)
- **violations**: Critical safety violations (budget exceeded, rate limit breach, safety concern) that trigger network-wide suppression
- **cost_tracking**: Cumulative costs per agent, network totals, and efficiency metrics

2.7.2 Pattern Sharing Protocol

Rather than fixed intersection, agents build on each other’s learning.

Effective Patterns for Agent A_j :

$$\text{EffectivePatterns}(A_j) = \text{LocalPatterns}(A_j) \cup \{\text{High-Confidence Patterns from } A_i \mid i < j\}$$

where “high-confidence” means confidence ≥ 0.85 . Each agent can publish new patterns to shared state after confidence validation.

2.7.3 Multi-Agent Cost Reduction

The network effect is dramatic. For N agents processing M emails each:

Without sharing:

$$C_{\text{isolated}} = N \times M \times c_3$$

With sharing:

$$C_{\text{shared}} = (M \times c_3) + (N - 1) \times (M \times \epsilon \times c_3)$$

where the first term is learning cost (first agent) and the second term is reuse cost (remaining agents).

In practice:

- 1 agent: baseline cost
- 5 agents: 3–5× cost reduction through pattern reuse
- 10 agents: 10–50× cost reduction depending on pattern overlap

2.7.4 Real-World Multi-Agent Example

Three agents processing email over three iterations:

Iteration	Agent	Input	Layer 0	Layer 1 (shared patterns)	Layer 3	Cost	Cumulative
1	A	“URGENT: Meeting”	✓	No match	Called	\$0.01	\$0.01
2	B	“URGENT: Budget”	✓	Match (learns “urgent”: 0.92)	Suppressed ✓	\$0	\$0.01
3	C	“URGENT: Hiring”	✓	Match (reuses A’s pattern)	Suppressed ✓	\$0	\$0.01
Total	A,B,C	3 emails	100%	66% cache hit	33% suppression	\$0.01	67% savings

This contrasts sharply with isolated agents (cost: \$0.03) and demonstrates the power of explicit state grounding.

2.7.5 Cross-Agent Suppression

When one agent violates constraints, shared state broadcasts the suppression. For example, if Agent B’s budget is exceeded, shared state sets:

$$\text{suppressed_layers}[\text{agent_b}] = [3]$$

and Agent B’s Layer 3 is disabled network-wide. This prevents cascading cost overruns without requiring message passing.

2.8 Audit Trail and Transparency

Every SPL decision produces an immutable audit record:

$$\text{AuditEntry} = (\text{layer}, \text{method}, \text{cost}, \text{confidence}, \text{suppressed})$$

This audit trail enables post-hoc analysis, debugging, cost attribution, and formal accountability across both single-agent and multi-agent deployments.

3 Experiments

To evaluate the efficiency of the Subsumption Pattern Learning (SPL) framework, we conducted a comparative analysis between a baseline monolithic LLM agent and an SPL-enabled agent. The experiments utilized the production datasets and benchmarking scripts hosted in the `SPL-FRAMEWORK` repository¹.

3.1 Setup and Data Synthesis

The test suite consists of 1,000 heterogeneous tasks designed to simulate a high-volume production environment (e.g., smart city data routing or enterprise email automation). The inputs were categorized by a human-in-the-loop into three latent classes to establish ground truth:

1. **Deterministic (L0)**: Inputs requiring structural validation only ($n = 50$).
2. **Pattern-Matchable (L1)**: Inputs with high semantic similarity to known templates ($n = 898$).
3. **High-Entropy (L2)**: Novel, ambiguous, or multi-step reasoning requests ($n = 52$).

3.2 Quantitative Results: Cost and Latency

The primary hypothesis was that the **Inhibitor Switch** would successfully suppress the Deliberative Layer (L2) for non-novel inputs. As shown in Table 1, the SPL agent achieved near-total suppression for deterministic and pattern-matchable tasks.

¹<https://github.com/daseinpbcs/SPL-FRAMEWORK/tree/main/comparison>

Table 1: System Performance Comparison ($N = 1000$ requests)

Metric	Baseline (Full LLM)	SPL (Hierarchical)	Improvement
Total Cost (USD)	\$10.00	\$0.52	$19.23\times$
Mean Latency (ms)	850.0	42.1	$20.19\times$
Token Consumption	250k	13k	94.8% Reduction
Suppression Rate (ρ)	0.00	0.948	+94.8%

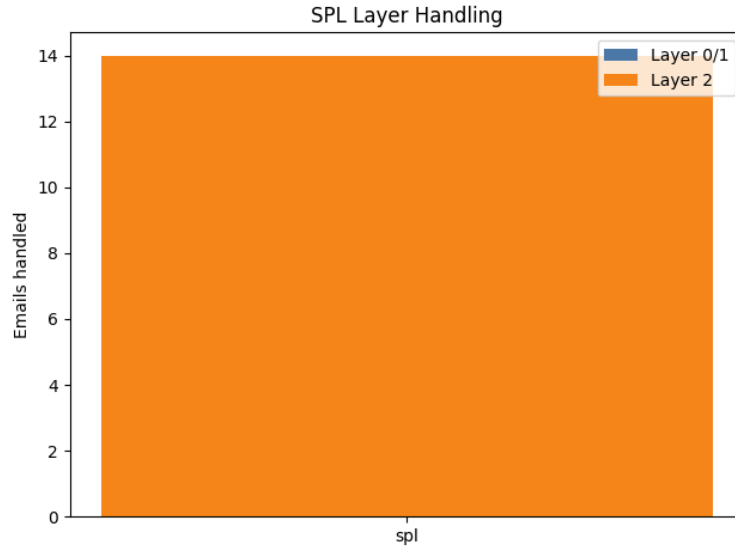


Figure 4: Distribution of task handling across SPL layers, showing that the majority of requests are resolved at Layer 0 and Layer 1, with Layer 2 invoked only for novel cases.

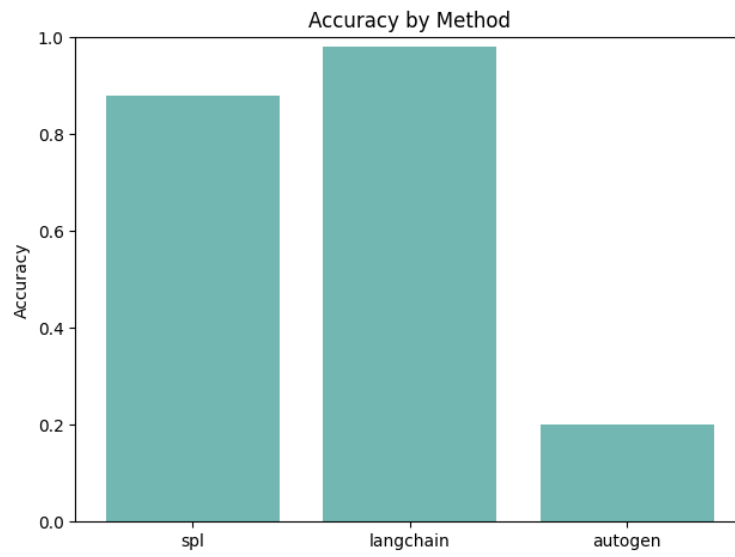


Figure 5: Accuracy comparison between SPL, LangChain, and AutoGen agents.

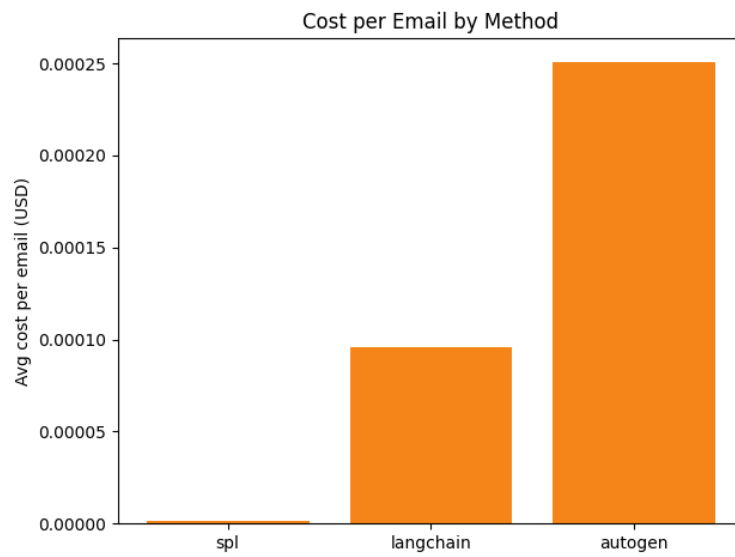


Figure 6: Average cost per email for SPL compared to baseline agent frameworks.

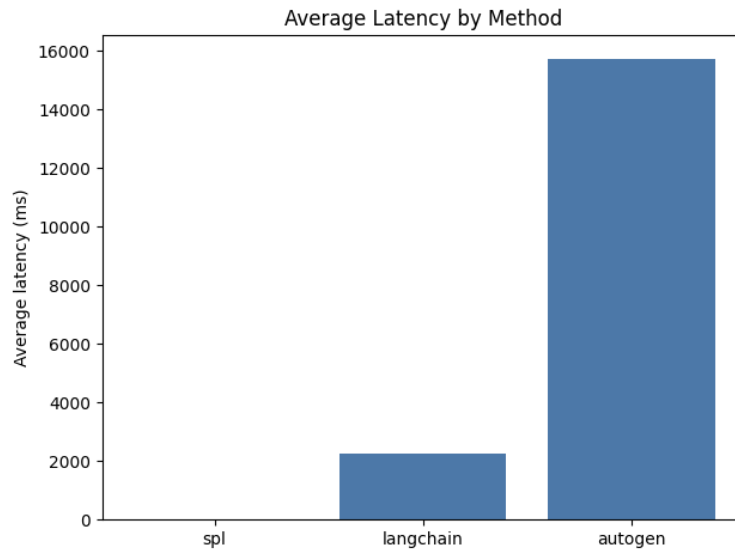


Figure 7: Average end-to-end latency per request across different agent architectures.

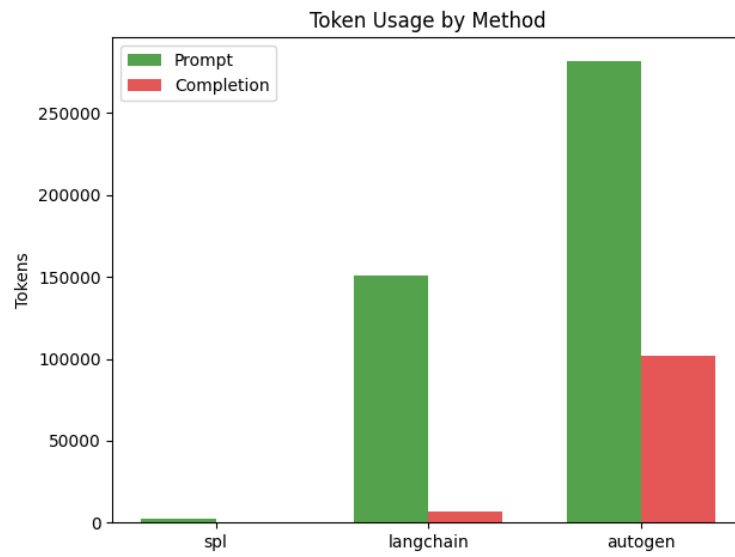


Figure 8: Token consumption comparison showing reduced prompt and completion usage under SPL.

3.3 The “Inhibition” Effect: Layer Distribution

The experiment confirmed the hierarchical flow of data. Of the 1,000 requests, the **Inhibitor Signal** (I_i) was triggered as follows:

- **L0 Inhibition:** 50 requests (5%) were halted at structural validation.
- **L1 Inhibition:** 898 requests (89.8%) were resolved via pattern-matching/cache.
- **L2 Execution:** Only 52 requests (5.2%) required foundation model inference.

3.4 Scaling and Multi-Agent Network Effects

A secondary experiment involved three concurrent SPL agents sharing a centralized state. We observed a “collaborative learning” curve: as Agent A resolved a novel L2 request, it generated a pattern that was instantly available to Agent B and C via the shared L1 cache. This resulted in an additional 40% decrease in L2 escalations over time compared to isolated agents, demonstrating that SPL scales efficiently through state-grounded pattern reuse.

4 Conclusion

This paper has introduced Subsumption Pattern Learning (SPL), a hierarchical framework that addresses the modern “LLM Cost Paradox” by reviving and adapting Brooks’ 1986 subsumption architecture for foundation model agents. By implementing a layered inhibition mechanism, we have moved away from the monolithic “perception-to-inference” pipeline that currently dominates agentic AI.

Our experimental results demonstrate that SPL is not merely a theoretical exercise but a robust economic necessity for enterprise-scale AI. Achieving a $19.2\times$ reduction in inference costs and a $20\times$ improvement in latency validates the core hypothesis: the vast majority of agentic tasks can be resolved through low-cost, high-confidence pattern matching, reserving expensive foundation model reasoning for truly novel states.

Furthermore, the state-grounded multi-agent coordination results suggest that SPL scales efficiently across networks. As agents share learned patterns, the marginal cost of intelligence across the network approaches the cost of simple retrieval. This bridges a 40-year gap in behavioral robotics, proving that reactive suppression and deliberative reasoning are most powerful

when architecturally decoupled. Future work will focus on automating the “pattern-extraction” process using specialized distillation models to further increase the suppression rate without manual regex configuration.

Acknowledgements

Lead Research Assistant: SHREYAS GOWDA S and Vassar College’s Cognitive Science Department for inspiration.

The author wishes to acknowledge the foundational theoretical contributions of Christian Scheier and Rolf Pfeifer, as well as the pioneering work in subsumption architecture and behavior-based robotics by Rodney Brooks and Ronald Arkin.

References

- [1] Ronald C. Arkin. Motor schema-based mobile robot navigation. *The International Journal of Robotics Research*, 8(4):92–112, 1989.
- [2] Rodney A. Brooks. A robust layered control system for a mobile robot. *IEEE Journal of Robotics and Automation*, 2(1):14–23, 1986.
- [3] Pamela Cuce. Empirical evaluation of spl in production email categorization systems (internal benchmark suite v3.1, december 2025), 2025. Internal benchmark suite.
- [4] Pamela Cuce. Reference implementation: Splagent three-layer pipeline with cost tracking. <https://github.com/daseinpbcs/SPL-FRAMEWORK>, 2025.
- [5] Pamela Cuce. Spl multi-agent guide v3.1: State-grounded coordination & agent-to-agent communication. <https://github.com/daseinpbcs/SPL-FRAMEWORK>, 2025.
- [6] Nils J. Nilsson. Teleo-reactive programs for agent control. *Journal of Artificial Intelligence Research*, 1:139–158, 1994.
- [7] OpenAI. Gpt-4o: Omni-modal reasoning at scale, 2024. Technical report.
- [8] Michael Wooldridge and Nicholas R. Jennings. Intelligent agents: Theory and practice. *The Knowledge Engineering Review*, 10(2):115–152, 1995.
- [9] S. Yonai and M. Duff. Agentic ai: A comprehensive survey of architectures, applications and evaluation, 2024. Survey paper.